Nom & Prénom : **SONGMY NGUENANG Yannick Anthony**
Classe : **M1 – MeRSI**

COMPTE RENDU DU TP 2 – Programmation client-serveur avec gRPC

1. Définition du service dans le fichier .proto et implémentation du serveur en Java

```
syntax = "proto3";

package factorization;

service FactorizationService { /* Définition du service Factorization */
    rpc Factorize (FactorRequest) returns (FactorResponse); /* Mode requete-reponse */

    rpc FactorizeStream (stream FactorRequest) returns (stream FactorResponse); /* Mode flux bidirectionnelle */
}

message FactorRequest {
   int64 number = 1;

}

message FactorResponse {
   repeated int64 factors = 1;

}
```

Après compilation du fichier ci-dessus, nous avons généré nos classes. La première classe **FactorizationServer,** qui implémente le serveur, et **FactorizationServiceImpl,** qui implémente le service de factorisation.

```
package factorization;


import io.grpc.Server;
import io.grpc.ServerBuilder;
import io.grpc.stub.StreamObserver;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class FactorizationServer {
   public static void main(String[] args) throws IOException, InterruptedException {
       Server server = ServerBuilder.forPort(50052)
                .addService(new FactorizationServiceImpl())
                .build()
                .start();
      System.out.println("Serveur gRPC démarré sur le port 50052...");
      server.awaitTermination();
    }
}
```

À partir de ce code, nous créons un serveur qui écoute sur le port 50052. Notre serveur ici est un processus qui ne s'arrête que si nous le voulons. En d'autres termes, c'est un processus qui ne s'arrête pas. À ce serveur, nous avons ajouté notre service implémenté comme suit :

```
package factorization;

import io.grpc.stub.StreamObserver;
import factorization.FactorizationServiceGrpc;
import factorization.Factorization;

public class FactorizationServiceImpl extends FactorizationServiceGrpc.FactorizationServiceImplBase {

   @Override
   public void factorize(Factorization.FactorRequest request, StreamObserver<Factorization.FactorResponse>
responseObserver) {
      long number = request.getNumber();
      Factorization.FactorResponse.Builder response = Factorization.FactorResponse.newBuilder();

      for (long i = 2; i * i <= number; i++) {
            while (number % i == 0) {
                  response.addFactors(i);
                  number /= i;
            }
      }
      if (number > 1) {
             response.addFactors(number);
       }

      responseObserver.onNext(response.build());
      responseObserver.onCompleted();
   }
}
```

Une fois le nombre à factoriser est lu, nous créons notre constructeur de réponse (**response)** que nous utiliserons pour créer la réponse envoyée au client. Une fois tous les facteurs ajoutés à la réponse, nous utilisons notre objet **responseObserver** (c'est un objet de type **StreamObserver<>** qui gère la communication entre le client et le serveur) pour créer la réponse finale (**FactorResponse).** Après l'envoi de notre réponse, nous indiquons que la communication est terminée (**responseObserver.onCompleted()).**

2. Implémentation du client gRPC

Nous implémenterons notre client avec le langage de programmation Python afin de garantir l'interopérabilité de notre application.

Une fois les fichiers **factorization_pb2.py** et **factorization_pb2_grpc.py** générés, nous pouvons implémenter notre client (client.py). Ces fichiers contiennent respectivement la définition des messages et les stubs pour permettant d'interagir avec le service gRPC.

L'implémentation du client est donnée par le bout de code suivant :

```
import grpc
import factorization_pb2
import factorization_pb2_grpc

def run():
    with grpc.insecure_channel("localhost:50052") as channel:
        stub = factorization_pb2_grpc.FactorizationServiceStub(channel)
        number = int(input("Entrez un nombre à factoriser : "))
        request = factorization_pb2.FactorRequest(number=number)
        response = stub.Factorize(request)
        print(f"Facteurs premiers de {number} : {list(response.factors)}")

if __name__ == "__main__":

    run()
```

Pour communiquer avec notre serveur situé en local et accessible via le port 50052, le client établit une connexion (ici, non sécurisée) pour se connecter. Tant que la connexion est établie, nous créons donc le stub qui nous permet de faire appel à nos méthodes. Une fois le service atteint, nous envoyons notre requête. Grâce à notre stub, nous accédons à la réponse fournie par le serveur.

```
anthony@Anthony:/mnt/d/Etude/Master 1-ENSI/Semestre 2/Systèmes Reparties et Nouvelles Technologie/grpc/server$ ./gradlew run &
[1] 147
anthony@Anthony:/mnt/d/Etude/Master 1-ENSI/Semestre 2/Systèmes Reparties et Nouvelles Technologie/grpc/server$ Starting a Gradle Daemon (subsequent
builds will be faster)

> IDLE
```

Figure 1 - Lancement du server Grpc

Sur u autre terminal, testons notre client.

```
anthony@Anthony:/mnt/d/Etude/Master 1-ENSI/Semestre 2/Systèmes Reparties et Nouvelles Technologie/grpc/client$ python3 client.py
Entrez un nombre à factoriser : 100
Facteurs premiers de 100 : [2, 2, 5, 5]
anthony@Anthony:/mnt/d/Etude/Master 1-ENSI/Semestre 2/Systèmes Reparties et Nouvelles Technologie/grpc/client$
```

Figure 2 - Test de notre service

Maintenant que la communication client-serveur classique fonctionne, nous allons modifier nos codes afin de les adapter pour l'implémentation d'une communication avec flux bidirectionnel.

Puisque dès le départ, nous avons spécifié dans notre fichier **factorization.proto** les deux modes de communication, il ne nous reste plus qu'à modifier l'implémentation de notre service et l'implémentation du client. Pour cela, nous modifions le code de **FactorizationServiceImpl.java** comme suit :

```
@Override
public StreamObserver<Factorization.FactorRequest>
factorizeStream(StreamObserver<Factorization.FactorResponse> responseObserver) {
    return new StreamObserver<Factorization.FactorRequest>() {
      @Override
      public void onNext(Factorization.FactorRequest request) {
        long number = request.getNumber();
        Factorization.FactorResponse.Builder response = Factorization.FactorResponse.newBuilder();

        for (long i = 2; i * i <= number; i++) {
          while (number % i == 0) {
              response.addFactors(i);
              number /= i;
           }
        }
        if (number > 1) {
             response.addFactors(number);
         }
         responseObserver.onNext(response.build()); // Envoie la réponse immédiatement
      }

      @Override
       public void onError(Throwable t) {
        System.err.println("Erreur lors du traitement : " + t.getMessage());
       }

      @Override
       public void onCompleted() {
         responseObserver.onCompleted(); // Fermeture du stream
       }
    };
}
```

L'ajout de la fonction **factorizeStream** qui retourne un **StreamObserver** permet au serveur d'attendre plusieurs requêtes client à la fois. Dans ce cas, le serveur n'attend pas la fin de la communication mais répond au fur et à mesure qu'il reçoit les requêtes du client **(responseObserver.onNext(response.build)).** Lorsque le client termine ses requêtes, la communication s'achève.

Du côté client, nous avons ajouté ce bout de code :

```
def generate_requests():
  while True:
    number = int(input("Entrez un nombre à factoriser (ou 0 pour quitter) : "))
    if number == 0:
      break
    yield factorization_pb2.FactorRequest(number=number)
```

C’est la construction d’un générateur qui envoie immédiatement au serveur les nombres à factoriser au fur et à mesure qu’ils sont saisis jusqu’à ce qu’il entre 0.

```
def run():
    with grpc.insecure_channel("localhost:50052") as channel:
        stub = factorization_pb2_grpc.FactorizationServiceStub(channel)
        response = stub.FactorizeStream(generate_requests())
        for reponse in responses:
            print(f"Facteurs premiers de {number} : {list(response.factors)}")
```

Le client peut désormais envoyer des nombres un par un au serveur qui renvoie les réponses immédiatement. Il est également capable d’écouter les réponses du serveur au fur et à mesure qu’elles sont envoyées.

```
anthony@Anthony: /mnt/d/E

anthony@Anthony:/mnt/d/Etude/Master 1-ENSI/Semestre 2/Systèmes Reparties et Nouvelles Technologie/grpc_/client$ python3 client.py
Entrez un nombre à factoriser (ou 0 pour quitter) : 100
Entrez un nombre à factoriser (ou 0 pour quitter) : Facteurs premiers : [2, 2, 5, 5]
100
Entrez un nombre à factoriser (ou 0 pour quitter) : Facteurs premiers : [2, 2, 5, 5]
200
Entrez un nombre à factoriser (ou 0 pour quitter) : Facteurs premiers : [2, 2, 2, 5, 5]
300
Entrez un nombre à factoriser (ou 0 pour quitter) : Facteurs premiers : [2, 2, 3, 5, 5]
0
anthony@Anthony:/mnt/d/Etude/Master 1-ENSI/Semestre 2/Systèmes Reparties et Nouvelles Technologie/grpc_/client$
```

Figure 3 - Test du service avec flux bidirectionnel

Notre service fonctionne correctement car nous arrivons à donner des nombres un à un au serveur et nous obtenons leur factorisation immédiatement. Notre client ne s’arrête que lorsqu’il tape 0.